Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
УРАЛЬСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ
ИМЕНИ ПЕРВОГО ПРЕЗИДЕНТА РОССИИ Б.Н. ЕЛЬЦИНА
(УрФУ имени первого Президента России Б.Н. Ельцина)
Институт радиоэлектроники и информационных технологий — РТФ
Школа профессионального и академического образования

ВВЕДЕНИЕ В МНОГОПОТОЧНОСТЬ

Отчет по лабораторной работе №6

по дисциплине «Программирование на Java»

	Дата	Подпись	
Преподаватель:	_____	_____	Архипов Н.А.
Студенты:	_____	_____	Пантелеев Е.А.

Группа: РИМ-150950

Екатеринбург

2026

ЦЕЛЬ РАБОТЫ

Получить представление о механизме многопоточности в языке программирования Java.

ХОД РАБОТЫ

1. Напишите программу, которая запускает два потока и выводит на экран их имена и текущее время в течение 10 секунд.

```
lab6 > Ex1.java > Ex1 > main(String[])
1  package lab6;
2  import java.time.LocalTime;
3
   GigaCode: explain | explain step by step | doc | test
4  public class Ex1 {
       Run | Debug
5      public static void main(String[] args) throws InterruptedException {
6          Thread t1 = new Thread(() -> {
7              for (int i = 0; i < 10; i++) {
8                  System.out.println(Thread.currentThread().getName() + ":" + LocalTime.now());
9                  try {
10                     Thread.sleep(millis: 1000);
11                 } catch (InterruptedException e) {
12                     e.printStackTrace();
13                 }
14             }
15         });
16
17         Thread t2 = new Thread(() -> {
18             for (int i = 0; i < 10; i++) {
19                 System.out.println(Thread.currentThread().getName() + ":" + LocalTime.now());
20                 try {
21                     Thread.sleep(millis: 1000);
22                 } catch (InterruptedException e) {
23                     e.printStackTrace();
24                 }
25             }
26         });
27
28         t1.start();
29         t2.start();
30
31         t1.join();
32         t2.join();
33     }
34 }
35
```

```
highs@LAPTOP-HS5F34LQ MINGW64 /c/java_ex (master)
$ java lab6/Ex1.java
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
Thread-1:17:33:09.691139500
Thread-0:17:33:09.691139500
Thread-1:17:33:10.704700700
Thread-0:17:33:10.711700
Thread-1:17:33:11.715185100
Thread-0:17:33:11.715185100
Thread-1:17:33:12.715962600
Thread-0:17:33:12.728960300
Thread-0:17:33:13.730591700
Thread-1:17:33:13.730591700
Thread-0:17:33:14.731435
Thread-1:17:33:14.734468500
Thread-0:17:33:15.739235400
Thread-1:17:33:15.739235400
Thread-0:17:33:16.740035100
Thread-1:17:33:16.744030700
Thread-1:17:33:17.749207500
Thread-0:17:33:17.749207500
Thread-1:17:33:18.749747400
Thread-0:17:33:18.764748
```

2. Напишите программу, которая запускает поток и выводит на экран числа от 1 до 10 с задержкой в 1 секунду между выводами.

```
lab6 > Ex2.java > Ex2 > main(String[])
package lab6;

GigaCode: explain | explain step by step | doc | test
public class Ex2 {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                System.out.println(i);
                try {
                    Thread.sleep(millis: 1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });

        t.start();
        t.join();
    }
}
```

```
highs@LAPTOP-HS5F34LQ MINGW64 /c/java_ex (master)
$ java lab6/Ex2.java
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
1
2
3
4
5
6
7
8
9
10
```

3. Напишите программу, которая создает два потока, которые выводят на экран числа от 1 до 10. Первый поток должен выводить только четные числа, а второй поток должен выводить только нечетные числа.

```
lab6 > Ex3.java > Ex3
package lab6;

GigaCode: explain | explain step by step | doc | test
public class Ex3 {
    private static final Object lock = new Object();
    private static int number = 1;

    Run | Debug
    public static void main(String[] args) {
        Thread evenThread = new Thread(() -> {
            synchronized (lock) {
                while (number < 10) {
                    if (number % 2 == 0) {
                        System.out.println(Thread.currentThread().getName() + ":" + number);
                        number++;
                    } else {
                        try {
                            lock.wait();
                        } catch (InterruptedException e) {
                            e.printStackTrace();
                        }
                    }
                    lock.notifyAll();
                }
            }
        });

        Thread oddThread = new Thread(() -> {
            synchronized (lock) {
                while (number < 10) {
                    if (number % 2 != 0) {
                        System.out.println(Thread.currentThread().getName() + ":" + number);
                        number++;
                    } else {
                        try {
                            lock.wait();
                        } catch (InterruptedException e) {
                            e.printStackTrace();
```

```
highs@LAPTOP-HS5F34LQ MINGW64 /c/java_ex (master)
$ java lab6/Ex3.java
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
Odd Thread:1
Even Thread:2
Odd Thread:3
Even Thread:4
Odd Thread:5
Even Thread:6
Odd Thread:7
Even Thread:8
Odd Thread:9
```

4. Напишите программу, которая создает 10 потоков и каждый поток выводит на экран свой номер.

```
lab6 > Ex4.java > Ex4
package lab6;

GigaCode: explain | explain step by step | doc | test
public class Ex4 {
    Run | Debug
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            final int threadNumber = i;
            Thread thread = new Thread(() -> {
                System.out.println("Thread " + threadNumber);
            });
            thread.start();
        }
    }
}
```

```
highs@LAPTOP-HS5F34LQ MINGW64 /c/java_ex (master)
$ java lab6/Ex4.java
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
Thread 1
Thread 6
Thread 9
Thread 7
Thread 5
Thread 10
Thread 2
Thread 3
Thread 4
Thread 8
```

5. Напишите функцию, которая находит максимальный элемент в массиве целых чисел при помощи многопоточности. Количество потоков должно быть равно количеству ядер процессора.

```
lab6 > Ex5.java > ...
GigaCode: explain | explain step by step | doc | test
public class Ex5 {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        int[] array = {3, 8, 1, 9, 5, 2, 7, 6, 10, 4};

        int max = findMaxParallel(array);
        System.out.println("Max: " + max);
    }

    public static int findMaxParallel(int[] array) throws InterruptedException {
        int cores = Runtime.getRuntime().availableProcessors();
        Thread[] threads = new Thread[cores];
        int[] partialMax = new int[cores];

        int chunkSize = (int) Math.ceil((double) array.length / cores);

        for (int i = 0; i < cores; i++) {
            final int threadIndex = i;
            final int start = i * chunkSize;
            final int end = Math.min(start + chunkSize, array.length);

            threads[i] = new Thread(() -> {
                int max = Integer.MIN_VALUE;
                for (int j = start; j < end; j++) {
                    if (array[j] > max) {
                        max = array[j];
                    }
                }
                partialMax[threadIndex] = max;
            });
            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
        }
```

```
highs@LAPTOP-HS5F34LQ MINGW64 /c/java_ex (master)
$ java lab6/Ex5.java
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
Max: 10
```

6. Напишите функцию, которая суммирует элементы в массиве целых чисел при помощи многопоточности. Количество потоков должно быть равно количеству ядер процессора.

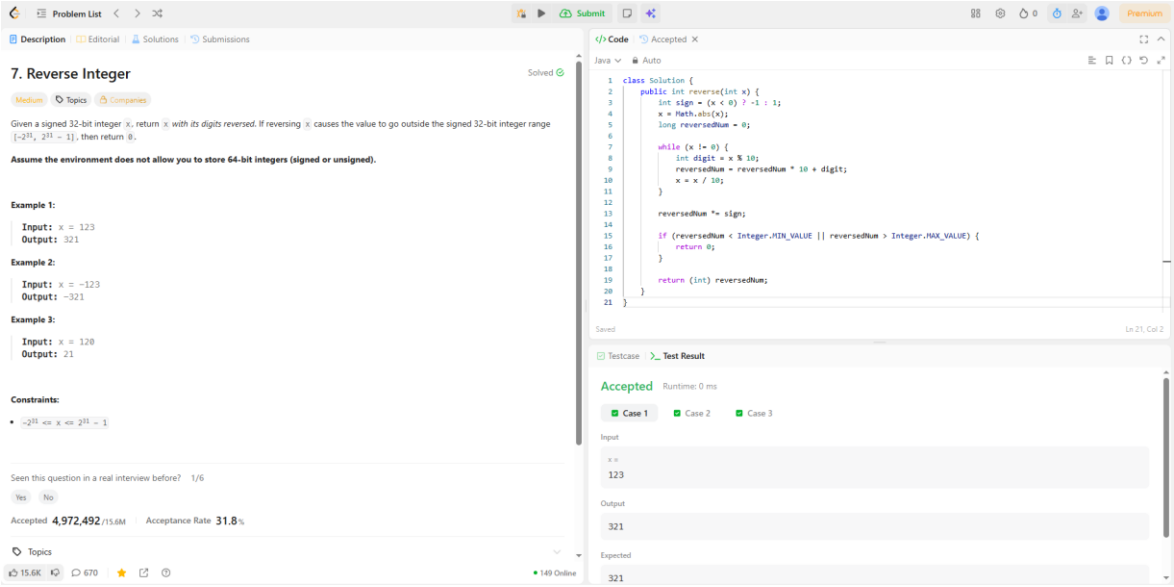
```
lab6 > Ex6.java > Ex6
public class Ex6 {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        int[] array = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

        int sum = sumParallel(array);
        System.out.println("Sum: " + sum);
    }

    public static int sumParallel(int[] array) throws InterruptedException {
        int cores = Runtime.getRuntime().availableProcessors();
        Thread[] threads = new Thread[cores];
        int[] partialSums = new int[cores];

        int chunkSize = (int) Math.ceil((double) array.length / cores);

        for (int i = 0; i < cores; i++) {
            final int threadIndex = i;
            final int start = i * chunkSize;
            final int end = Math.min(start + chunkSize, array.length);

            threads[i] = new Thread(() -> {
                int sum = 0;
                for (int j = start; j < end; j++) {
                    sum += array[j];
                }
                partialSums[threadIndex] = sum;
            });
            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
        }

        int totalSum = 0;
        for (int sum : partialSums) {
```

```
highs@LAPTOP-HS5F34LQ MINGW64 /c/java_ex (master)
$ java lab6/Ex6.java
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
Sum: 55
```

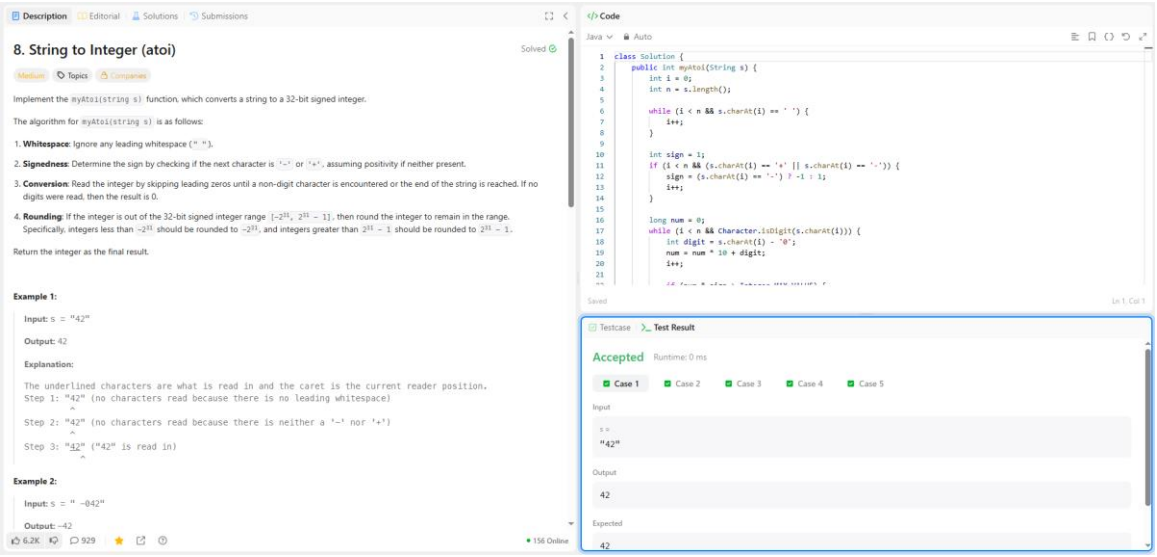
Со всем кодом в заданиях лучше ознакомиться на github.

7. Задачи Leetcode

Задача 7. Reverse Integer



Задача 8. String to Integer (atoi)



Ссылка на github

https://github.com/Fothiss/java_apps

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы было получено представление о механизме многопоточности в языке программирования Java.